

23IT204T – DATABASE MANAGEMENT SYSTEM

UNIT 1: RELATIONAL DATABASES

PURPOSE OF DBMS:

Data Management and Organization: A DBMS provides a systematic way to store, retrieve, and manage data in an organized manner, making it easy for users to access and manipulate large amounts of data efficiently.

Data Integrity and Security: It enforces data integrity constraints to ensure data accuracy and consistency. It also provides security mechanisms to control data access and protect sensitive information from unauthorized users.

Data Sharing and Multi-user Support: DBMS allows multiple users to access and work with the data concurrently while maintaining data consistency and isolation, which is essential for collaborative environments.

Data Recovery and Backup: It includes features for data recovery and backup, ensuring that data can be restored in case of accidental loss, hardware failure, or other issues, which maintains the availability and reliability of data.

Efficient Query Processing: DBMS enables efficient data retrieval using structured query languages (like SQL), allowing users to extract and analyze data easily, supporting decision-making and reporting tasks.

VIEWS OF DATA:

In DBMS, the concept of **views of data** refers to the different levels of abstraction that allow users to interact with data in various ways, based on their needs and roles. This concept is foundational in the architecture of a DBMS and is structured into three levels.

1. **Physical Level (Internal Level):**

- This level represents how data is actually stored in the database, focusing on storage, data structures, and file organization. It deals with physical storage details and performance tuning but hides these complexities from users.
- Example: How data is arranged on disks and how index structures are implemented for efficient retrieval.

2. **Logical Level (Conceptual Level):**

- The logical level provides an abstract view of the entire database, capturing the relationships among data without specifying how data is stored. It focuses on the structure and meaning of the data, rather than the technical details of storage.
- Example: Tables, attributes, and relationships among tables in an entity-relationship model, showing how customer, order, and product data relate.

3. **View Level (External Level):**

- This level is concerned with how individual users interact with the database. Different users may have different views depending on their role and need for data. It presents only a subset of the database relevant to a specific user group, offering data security and simplicity.
- Example: A sales team might only see customer names, orders, and prices, while an HR team might see employee information without access to sales data.

DATA MODELS:

Data models in DBMS provide a framework for organizing, storing, and defining data relationships. They are essential for shaping how data is stored, retrieved, and managed within a database.

1. Hierarchical Data Model:

- Data is organized in a tree-like structure where each record has a single parent, but a parent can have multiple children. This model is suitable for applications with one-to-many relationships.
- **Example:** A file system where folders contain subfolders and files, or an organizational chart with departments and sub-departments.

2. Network Data Model:

- This model allows for a more flexible structure than the hierarchical model, where a record can have multiple parent and child records, forming a graph-like model.
- **Example:** A network of interconnected records, such as a product component hierarchy where a component can belong to multiple products.

3. Relational Data Model:

- The relational model organizes data into tables (relations) with rows (tuples) and columns (attributes). It is based on set theory and uses a structured query language (SQL) for data manipulation.
- **Example:** A database with customer, order, and product tables that relate through keys, making it possible to query and relate data across tables flexibly.

4. Entity-Relationship (ER) Model:

- The ER model focuses on entities (objects) and relationships between them, usually represented visually with an ER diagram. It helps in the conceptual design phase of databases by modeling real-world entities and their interactions.
- **Example:** An ER diagram modeling an online bookstore, with entities like *Customer*, *Book*, and *Order*, and relationships between them such as "places" or "contains."

5. Object-Oriented Data Model:

- This model combines object-oriented programming principles with database management, where data and behavior are encapsulated in objects. It's used in applications that require complex data structures and interactions.
- **Example:** A multimedia database, where images, audio, and video are stored as objects with associated methods for processing.

6. Document Data Model (NoSQL):

- Part of the NoSQL family, this model stores data in document formats like JSON or BSON, ideal for semi-structured and unstructured data. Each document can have a different structure, allowing for flexibility.
- **Example:** A product catalog in an e-commerce application where each product document may have different fields based on the product type.

DATABASE SYSTEM ARCHITECTURE:

1. Two-Tier Architecture

- **Client-Server Model:** This architecture involves a client application directly communicating with the database server.
- **Client Side:** The client side handles the user interface and business logic. It connects to the database server over a network to send queries and receive results.
- **Server Side:** The server side (DBMS) processes the queries, performs operations on the database, and returns the results to the client.
- **Usage:** Often used in small-scale applications where a simple, direct interaction between client and server is sufficient.

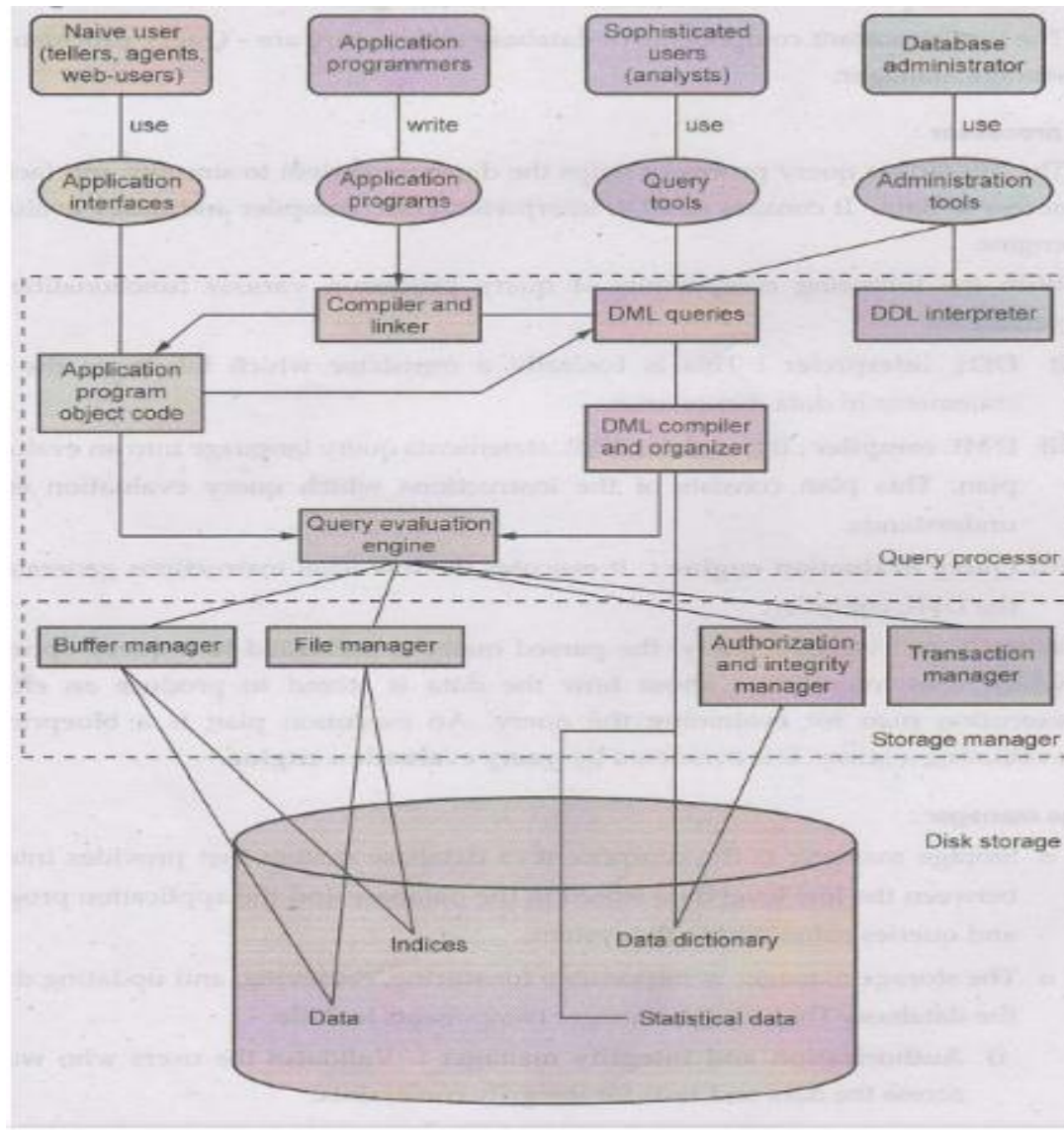
2. Three-Tier Architecture

- This is a more sophisticated structure, typically used in web applications or distributed databases, where there are three layers:
 - **Presentation Layer (Client Tier):** The user interface, responsible for interacting with the end-users, is hosted on client devices. This layer often includes web browsers or desktop applications.
 - **Application Layer (Middle Tier):** This layer hosts the application server, where business logic is processed. It acts as an intermediary between the client and database, sending queries to the database and processing data for presentation.
 - **Database Layer (Data Tier):** This layer includes the DBMS that stores and retrieves data. It handles database access, query processing, and data storage management.
- **Advantages:** This architecture enhances security, scalability, and data integrity since the client only accesses the application server and not the database directly.

3. N-Tier Architecture

- In large-scale or cloud-based systems, additional layers may be introduced, creating an N-tier architecture where multiple servers or components may handle specialized tasks, like load balancing, caching, or API management.
- **Benefits:** Greater flexibility, scalability, and distribution of resources. Each layer can be managed, scaled, and updated independently.

DATABASE SYSTEM ARCHITECTURE:



- Consider the top part of Fig. 1.5.1. It shows application interfaces used by naïve users, application programs created by application programmers, query tools used by sophisticated users and administration tools used by database administrator.
- The lowest part of the architecture is for disk storage.
- The two important components of database architecture are - Query processor and storage manager.

Query processor:

- The interactive query processor helps the database system to simplify and facilitate access to data. It consists of DDL interpreter, DML compiler and query evaluation engine.
- With the following components of query processor, various functionalities are performed -
 - i) DDL interpreter: This is basically a translator which interprets the DDL statements in data dictionaries.
 - ii) DML compiler: It translates DML statements query language into an evaluation plan. This plan consists of the instructions which query evaluation engine understands.
 - iii) Query evaluation engine: It executes the low-level instructions generated by the DML compiler.
- When a user issues a query, the parsed query is presented to a query optimizer, which uses information about how the data is stored to produce an efficient execution plan for evaluating the query. An execution plan is a blueprint for evaluating a query. It is evaluated by query evaluation engine.

Storage manager:

- Storage manager is the component of database system that provides interface between the low level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible for storing, retrieving, and updating data in the database. The storage manager components include -
 - i) Authorization and integrity manager: Validates the users who want to access the data and tests for integrity constraints.
 - ii) Transaction manager: Ensures that the database remains in consistent despite of system failures and concurrent transaction execution proceeds without conflicting.
 - iii) File manager: Manages allocation of space on disk storage and representation of the information on disk.
 - iv) Buffer manager: Manages the fetching of data from disk storage into main memory. The buffer manager also decides what data to cache in main memory. Buffer manager is a crucial part of database system.
- Storage manager implements several data structures such as -
 - i) Data files: Used for storing database itself.

- ii) Data dictionary: Used for storing metadata, particularly schema of database.
- iii) Indices: Indices are used to provide fast access to data items present in the database

User Level:

Naïve Users: Naïve users who interact with system by working one of the application program written previously.

Application Programmers: They are the computer professionals who write application programs.

Sophisticated Users(Programmers): They interact with system by forming the request in a database query language.

Database Administrator(DBA): A Person who has central control over entire database is called as database Administrator.

Functions of DBA:

- Schema Definition
- Schema and Physical organization and modification
- Granting of authorization for data access
- Routine Maintenance

INTRODUCTION TO RELATIONAL DATABASES:

Relational databases are a type of database based on the relational model proposed by E. F. Codd in 1970. This model organizes data into structured tables, or "relations," which makes it easy to store, manage, and query data. Relational databases are widely used in applications because of their simplicity, flexibility, and ability to handle large volumes of structured data. Here's an introduction to the core concepts of relational databases, as discussed in the works of Ramakrishna, Elmasri, and Silberschatz:

Key Concepts of Relational Databases

1. Tables (Relations):

- In relational databases, data is stored in tables, each representing an entity or concept (e.g., *Customers*, *Orders*, *Products*).
- A table has rows (records or tuples) representing individual instances of the entity and columns (attributes) representing the properties or fields of that entity.

2. Rows (Tuples):

- Each row in a table represents a single, unique entry, such as a specific customer, order, or product.
- A tuple consists of attribute values that collectively describe that particular instance.

3. Columns (Attributes):

- Each column in a table represents an attribute, which is a characteristic of the entity. For example, in a *Customer* table, columns might include *CustomerID*, *Name*, *Email*, and *Address*.
- Attributes define the type of data that can be stored in a particular column, such as integer, string, or date.

4. Primary Key:

- Each table typically has a primary key, a unique identifier for each record in the table. This key ensures that each row is uniquely identifiable, preventing duplicate entries.
- **Example:** In a *Customer* table, *CustomerID* could be the primary key.

5. Foreign Key:

- A foreign key is an attribute in one table that links to the primary key of another table, establishing a relationship between the two tables.
- **Example:** An *Order* table may have a *CustomerID* foreign key that references the *CustomerID* primary key in the *Customer* table, associating each order with a specific customer.

6. Relationships:

- Relational databases allow relationships between tables, which can be:
 - **One-to-One:** Each row in one table relates to only one row in another.
 - **One-to-Many:** One row in a table can relate to multiple rows in another (e.g., a customer may place multiple orders).
 - **Many-to-Many:** Rows in one table can relate to multiple rows in another (e.g., students and courses).

7. Structured Query Language (SQL):

- SQL is the standard language used for interacting with relational databases. It enables users to create, read, update, and delete data (CRUD operations) and perform complex queries.
- SQL operations include *SELECT* for retrieving data, *INSERT* for adding new data, *UPDATE* for modifying data, and *DELETE* for removing data.

Advantages of Relational Databases

- **Data Integrity:** With rules such as primary keys, foreign keys, and constraints, relational databases enforce data accuracy and integrity.
- **Flexibility:** Relational databases can handle large volumes of data and complex queries, making them ideal for diverse applications.
- **Data Consistency:** The use of ACID (Atomicity, Consistency, Isolation, Durability) properties ensures transactions are reliably processed, even in multi-user environments.
- **Data Security:** Access control can be implemented at various levels, allowing for secure, role-based data access.

Applications

Relational databases are widely used in domains requiring structured, consistent data, such as banking, e-commerce, healthcare, and enterprise resource planning (ERP) systems. Their ability

to manage complex data relationships and ensure data integrity has made them the backbone of many enterprise and business applications.

KEYS IN DBMS:

In relational databases, *keys* are essential elements that help in identifying, organizing, and maintaining relationships between records in tables. Keys enforce uniqueness and facilitate efficient data retrieval and linking across tables.

Types of Keys in Relational Databases

In the context of relational databases, **keys** are critical elements that uniquely identify records, establish relationships, and ensure data integrity. The concept of keys is central to relational database design, as they help organize and maintain consistency across tables.

1. Primary Key

- **Definition:** A primary key is a unique identifier for each record (row) in a table. No two rows can have the same value for the primary key.
- **Characteristics:**
 - Uniqueness: Each record must have a unique primary key.
 - Non-null: A primary key cannot contain a NULL value.
 - Immutability: The value of the primary key should not change once assigned.
- **Example:** In a *Student* table, *StudentID* can be the primary key, ensuring each student has a unique ID.

2. Foreign Key

- **Definition:** A foreign key is an attribute (or a combination of attributes) in one table that links to the primary key in another table. It establishes a relationship between the two tables.
- **Characteristics:**
 - A foreign key may have duplicate values, meaning multiple rows in the referencing table can refer to the same row in the referenced table.
 - A foreign key can contain NULL values if the relationship allows for optional associations.
- **Example:** In an *Orders* table, the *CustomerID* field could be a foreign key that refers to the *CustomerID* primary key in a *Customers* table, linking each order to a specific customer.

3. Candidate Key

- **Definition:** A candidate key is a set of one or more attributes that can uniquely identify each record in a table. A table may have multiple candidate keys, and one of them is chosen as the primary key.
- **Characteristics:**

- Any candidate key can potentially be used as the primary key.
 - Candidate keys must be unique and minimal, meaning they cannot contain any unnecessary attributes.
- **Example:** In an *Employee* table, both *EmployeeID* and *Email* might be candidate keys, as each can uniquely identify an employee.

4. Alternate Key

- **Definition:** An alternate key is any candidate key that is not chosen as the primary key. It provides an alternative way of identifying records uniquely.
- **Characteristics:**
 - An alternate key must also be unique and non-null.
- **Example:** If *EmployeeID* is chosen as the primary key, then *Email* might be considered an alternate key for the *Employee* table.

5. Composite Key

- **Definition:** A composite key is a primary key that consists of more than one attribute. It is used when a single attribute is not sufficient to uniquely identify a record.
- **Characteristics:**
 - The combination of values in the attributes that form the composite key must be unique for each row.
- **Example:** In a *CourseRegistration* table, a composite key might consist of *StudentID* and *CourseID*, where each student can register for multiple courses, but the combination of student and course must be unique.

6. Superkey

- **Definition:** A superkey is a set of attributes that uniquely identifies each record in a table. A superkey may contain additional attributes beyond those necessary for uniqueness.
- **Characteristics:**
 - All primary keys are superkeys, but not all superkeys are primary keys. A superkey can contain extra attributes that are not required to ensure uniqueness.
- **Example:** If *StudentID* is the primary key for a *Student* table, then the combination of *StudentID* and *Name* can be a superkey, even though *Name* is not necessary for uniqueness.

7. Unique Key

- **Definition:** A unique key ensures that all values in a column (or a set of columns) are distinct, but unlike the primary key, it allows NULL values.
- **Characteristics:**
 - Unique keys ensure that the values in the columns are unique, but they can still contain NULLs (depending on the DBMS).
 - A table can have multiple unique keys.

- **Example:** In a *User* table, the *Username* field might have a unique constraint, ensuring that each username is distinct, but it can allow a NULL value.

8. Secondary Key

- **Definition:** A secondary key is an attribute or set of attributes used for data retrieval purposes, but it is not a part of the primary key.
- **Characteristics:**
 - Secondary keys help improve query performance, especially for searching or sorting data based on non-primary key attributes.
- **Example:** In a *Product* table, a secondary key might be the *ProductName* attribute, which can be used for quick searching but is not necessarily unique.

Summary of Key Types:

- **Primary Key:** Uniquely identifies a record and cannot contain NULLs.
- **Foreign Key:** Establishes a relationship between two tables.
- **Candidate Key:** A set of attributes that could serve as a primary key.
- **Alternate Key:** A candidate key that is not chosen as the primary key.
- **Composite Key:** A primary key formed by multiple attributes.
- **Superkey:** Any set of attributes that can uniquely identify records (including those with extra attributes).
- **Unique Key:** Ensures unique values but may allow NULLs.
- **Secondary Key:** Used for efficient data retrieval but not a primary key.

INTEGRITY CONSTRAINTS:

Integrity constraints in relational databases are rules that ensure the accuracy, consistency, and validity of the data stored in a database. These constraints enforce data integrity by limiting the types of data that can be inserted into tables and ensuring that the relationships between tables remain valid. Integrity constraints are crucial in maintaining the quality and reliability of the database. The main types of integrity constraints are as follows:

1. Domain Integrity Constraints

- **Definition:** These constraints ensure that the values entered into a column are of the correct data type, fall within an acceptable range, and adhere to predefined formats or standards.
- **Examples:**
 - If a column is defined to store integer values, only integers can be inserted.
 - If a column is defined to store dates, only valid date values are allowed.
 - If a column is defined with a range, such as an *age* column between 18 and 100, values outside this range would be disallowed.

```
CREATE TABLE Employee (EmployeeID INT, Name VARCHAR(100), Age INT
CHECK(Age >= 18 AND Age <= 100));
```

2. Entity Integrity Constraints

- **Definition:** These constraints ensure that each row (record) in a table can be uniquely identified, and that there is no duplication of records.
- **Primary Key Constraint:** The primary key must contain unique, non-null values for each record in the table.
 - **Uniqueness:** Ensures that no two records in the table have the same value for the primary key.
 - **Not-null:** Ensures that the primary key cannot have NULL values, as it is used to uniquely identify rows.
- **Example:** The EmployeeID column in the *Employee* table is defined as the primary key to guarantee that each employee has a unique identifier.

```
CREATE TABLE Employee (EmployeeID INT PRIMARY KEY, Name VARCHAR(100));
```

3. Referential Integrity Constraints

- **Definition:** These constraints ensure that foreign key relationships between tables are valid. A foreign key in one table must match a primary key in another table, or it must be NULL (if allowed).
- **Foreign Key Constraint:** This rule ensures that data in the foreign key column(s) corresponds to valid entries in the primary key column(s) of another table.
 - **Example:** A foreign key in the *Orders* table might reference the *CustomerID* primary key in the *Customers* table.
 - **Cascading Actions:** Referential integrity can include actions like "CASCADE" (where changes in the referenced table propagate to the referencing table) or "SET NULL" (where a foreign key is set to NULL if the referenced record is deleted).

```
CREATE TABLE Orders (  
  
    OrderID INT PRIMARY KEY,  
  
    CustomerID INT,  
  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
  
);
```

Referential actions:

- **ON DELETE CASCADE:** If a referenced row is deleted, the corresponding rows in the child table are also deleted.
- **ON UPDATE CASCADE:** If the referenced primary key value is updated, the corresponding foreign key values are updated.

```
CREATE TABLE Orders (
```

```

OrderID INT PRIMARY KEY,

CustomerID INT,

FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)

ON DELETE CASCADE

);

```

4. User-Defined Integrity Constraints

- **Definition:** These are custom rules that are defined by users to enforce business-specific conditions that cannot be captured by the built-in domain, entity, or referential constraints.
- **Example:** If a business rule requires that an employee's salary must always be greater than the minimum wage, a user-defined constraint can enforce this rule.

```

CREATE TABLE Employee (

EmployeeID INT PRIMARY KEY,

Name VARCHAR(100),

Salary DECIMAL(10, 2),

CHECK (Salary > 5000) -- Custom rule: Salary must be greater than 5000

);

```

5. NULL Integrity Constraints

- **Definition:** These constraints specify whether a column is allowed to contain NULL values (i.e., no data). Some columns must always have a value (NOT NULL), while others may be allowed to have no value (NULL).
- **NOT NULL:** Ensures that a column cannot have NULL values.
- **Example:** A *Name* column in the *Employee* table might require that every employee has a name (NOT NULL), while a *MiddleName* column might allow NULL values if not every employee has a middle name.

```

CREATE TABLE Employee (

EmployeeID INT PRIMARY KEY,

Name VARCHAR(100) NOT NULL,

```

MiddleName VARCHAR(100) NULL

);

6. Check Constraints

- **Definition:** A check constraint allows the database to enforce a condition that must be true for each row. It is used to limit the range of values that can be stored in a column.
- **Example:** Enforcing that the *Age* of an employee is between 18 and 100 years.

```
CREATE TABLE Employee (
```

```
    EmployeeID INT PRIMARY KEY,
```

```
    Name VARCHAR(100),
```

```
    Age INT,
```

```
    CHECK (Age BETWEEN 18 AND 100) -- Age must be between 18 and 100
```

```
);
```

Summary of Integrity Constraints:

- **Domain Integrity:** Ensures data is valid based on its type and range.
- **Entity Integrity:** Ensures that each record is uniquely identifiable and no nulls in primary keys.
- **Referential Integrity:** Ensures that relationships between tables are valid, with foreign keys properly linking to primary keys.
- **User-Defined Integrity:** Custom rules set by the user to enforce business logic.
- **NULL Integrity:** Specifies whether columns can have NULL values.
- **Check Constraints:** Enforces specific conditions on the data, like ranges or values.

RELATIONAL ALGEBRA:

Relational Algebra is a procedural query language used to query and manipulate relational databases. It provides a set of operations to retrieve, insert, and modify data in relational tables (relations) using mathematical set theory. Relational algebra is the foundation for SQL and other database query languages. It helps in expressing complex queries in a structured and systematic way.

1. **Select (σ):**

- The **Select** operation retrieves rows from a relation that satisfy a specified condition.
- Syntax: $\sigma_{\text{condition}}(R)$

- Example: $\sigma_{age > 25}(\text{Employees})$ selects all rows from the Employees relation where the age attribute is greater than 25.
- 2. **Project (π):**
 - The **Project** operation retrieves specific columns (attributes) from a relation, effectively reducing the number of attributes.
 - Syntax: $\pi_{\text{attribute1}, \text{attribute2}, \dots}(\text{R})$
 - Example: $\pi_{\text{name}, \text{age}}(\text{Employees})$ projects the name and age columns from the Employees relation.
- 3. **Union ($\cup$):**
 - The **Union** operation combines the tuples of two relations (which must have the same number and type of attributes) and returns all distinct tuples from both relations.
 - Syntax: $\text{R} \cup \text{S}$
 - Example: $\text{Employees} \cup \text{Contractors}$ gives a relation containing all unique tuples from both Employees and Contractors.
- 4. **Difference ($-$):**
 - The **Difference** operation retrieves the tuples that are in the first relation but not in the second.
 - Syntax: $\text{R} - \text{S}$
 - Example: $\text{Employees} - \text{Contractors}$ gives the tuples that are in the Employees relation but not in the Contractors relation.
- 5. **Cartesian Product ($\times$):**
 - The **Cartesian Product** operation combines each tuple of one relation with each tuple of another relation, resulting in a new relation with all possible pairs of tuples.
 - Syntax: $\text{R} \times \text{S}$
 - Example: $\text{Employees} \times \text{Departments}$ results in a relation where each employee is paired with every department.
- 6. **Rename (ρ):**
 - The **Rename** operation changes the name of a relation or its attributes.
 - Syntax: $\rho_{\text{new_relation_name}}(\text{R})$
 - Example: $\rho(\text{Emp}, \text{Employees})$ renames the Employees relation to Emp.
- 7. **Join ($\bowtie$):**
 - The **Join** operation combines tuples from two relations based on a common attribute. There are different types of joins, such as **Inner Join**, **Natural Join**, and **Theta Join**.
 - Syntax (for Natural Join): $\text{R} \bowtie \text{S}$
 - Example: $\text{Employees} \bowtie \text{Departments}$ performs a natural join between Employees and Departments, where the join is done on common attributes (e.g., department_id).

Derived Operations:

In addition to these basic operations, some other operations can be derived from combinations of the above operations, such as:

- **Intersection ($\cap$):** The set of tuples common to both relations (equivalent to $R \cap S = (R \cup S) - (R - S)$).
- **Division ($\div$):** Used to find tuples in one relation that are associated with all tuples in another relation (e.g., for queries like "which employees worked in all projects?").

Example Query in Relational Algebra:

Assume we have two relations:

- **Employees(Emp_ID, Name, Age, Dept_ID)**
- **Departments(Dept_ID, Dept_Name)**

To find the names of employees working in a specific department, we can use the following relational algebra operations:

1. **Select** employees from a department (e.g., "Sales"): $\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments})$
2. **Join** the result with the Employees table to get the names: $\pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments}) \bowtie \text{Employees})$

This query first selects the department "Sales," then joins the resulting relation with the Employees relation to obtain the names of employees in the Sales department.

Summary of Relational Algebra Operations:

1. **Selection (σ):** Filters rows based on a condition.
2. **Projection (π):** Selects specific columns from a table.
3. **Union ($\cup$):** Combines two relations and eliminates duplicates.
4. **Set Difference ($-$):** Returns rows in one relation but not in the other.
5. **Cartesian Product ($\times$):** Combines every row from one relation with every row from another.
6. **Rename (ρ):** Renames relations or attributes.
7. **Join ($\bowtie$):** Combines two relations based on a join condition.
8. **Division ($\div$):** Finds records that are related to all records in another relation.

Importance of Relational Algebra:

- **Foundation for SQL:** Relational Algebra forms the theoretical basis for SQL (Structured Query Language). SQL operations like SELECT, JOIN, and WHERE can be mapped to relational algebra operations.
- **Query Optimization:** Understanding relational algebra helps in designing and optimizing queries by choosing the most efficient execution plans.
- **Formal Query Language:** It provides a formal way to express queries, which is helpful in optimizing database queries.
- **Set-Based Operations:** Relational algebra is based on set theory, which allows for efficient handling of data in relational databases.

These operations are building blocks for formulating queries and understanding how data can be retrieved and manipulated in relational databases.

RELATIONAL ALGEBRA – EXAMPLES

Relations:

- **Employees**(Emp_ID, Name, Age, Dept_ID)
- **Departments**(Dept_ID, Dept_Name)

1. Find all employees' names and ages

- Query: List the Name and Age of all employees.
- **Relational Algebra:** $\pi_{\text{Name, Age}}(\text{Employees})$

2. Find all employees who are older than 30

- Query: Select employees whose age is greater than 30.
- **Relational Algebra:** $\sigma_{\text{Age} > 30}(\text{Employees})$

3. Find the names of all employees working in the "Sales" department

- Query: Find employees who work in the "Sales" department.
- **Relational Algebra:** $\pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments}) \bowtie \text{Employees})$
- Explanation: First, we select the "Sales" department from **Departments** using **Select** (σ). Then, we join it with the **Employees** relation on **Dept_ID**, and finally project the Name of employees using **Project** (π).

4. Find the names of employees who are in departments with less than 10 employees

- Query: List names of employees who work in a department that has fewer than 10 employees.
- **Relational Algebra:** $\pi_{\text{Name}}(\text{Employees} \bowtie (\pi_{\text{Dept_ID}}(\sigma_{\text{count}(\text{Emp_ID}) < 10}(\text{Employees}))))$
- Explanation: This query uses a **Join** and aggregates the employees based on the department.

5. Find all departments and the number of employees in each department

- Query: List each department along with the number of employees in that department.
- **Relational Algebra:** $\pi_{\text{Dept_Name, count}(\text{Emp_ID})}(\text{Employees} \bowtie \text{Departments})$

6. Find all employees who work in both the "HR" and "IT" departments

- Query: Find employees who work in both the "HR" and "IT" departments (assuming each department has its own row in the Employees table).

- **Relational Algebra:** $\pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'HR'}}(\text{Departments}) \bowtie \text{Employees}) \cap \pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'IT'}}(\text{Departments}) \bowtie \text{Employees})$
- Explanation: The intersection ($\cap$) operator returns employees who work in both "HR" and "IT."

7. Find all employees who do not belong to the "Sales" department

- Query: List employees who do not work in the "Sales" department.
- **Relational Algebra:** $\text{Employees} - \pi_{\text{Emp_ID}}(\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments}) \bowtie \text{Employees})$
- Explanation: We first join **Departments** with **Employees** to select those in the "Sales" department and subtract that set from all employees using the **Difference** ($-$) operation.

8. Find employees with the same department as "John Doe"

- Query: Find all employees working in the same department as "John Doe."
- **Relational Algebra:** $\pi_{\text{Name}}(\sigma_{\text{Dept_ID} = (\pi_{\text{Dept_ID}}(\sigma_{\text{Name} = \text{'John Doe'}}(\text{Employees})))}(\text{Employees}))$
- Explanation: First, we select the Dept_ID for "John Doe" and use it to filter all employees in the same department.

9. List employees who are either in the "Sales" or "HR" department

- Query: List employees working in either the "Sales" or "HR" department.
- **Relational Algebra:** $\pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments}) \bowtie \text{Employees}) \cup \pi_{\text{Name}}(\sigma_{\text{Dept_Name} = \text{'HR'}}(\text{Departments}) \bowtie \text{Employees})$
- Explanation: The **Union** ($\cup$) operator combines the results of two queries, one for the "Sales" department and one for the "HR" department.

10. Find employees who do not work in the "Sales" department

- Query: List employees who do not work in the "Sales" department.
- **Relational Algebra:** $\text{Employees} - \pi_{\text{Emp_ID}}(\sigma_{\text{Dept_Name} = \text{'Sales'}}(\text{Departments}) \bowtie \text{Employees})$
- Explanation: This query subtracts the employees in the "Sales" department from all employees using the **Difference** ($-$) operation.

11. Find the employees with the highest salary in each department

- Query: List the employees who earn the highest salary in their respective departments.
- **Relational Algebra:** $\pi_{\text{Emp_ID}, \text{Name}, \text{Salary}}(\sigma_{\text{Salary} = \text{max}(\text{Salary})}(\text{Employees}))$
- Explanation: This uses a **Select** (σ) to filter employees who earn the maximum salary in each department.

12. Find the department names that have employees with an age greater than 40

- Query: List the departments where employees are older than 40.
- **Relational Algebra:** $\pi_{\text{Dept_Name}}(\sigma_{\text{Age} > 40}(\text{Employees}) \bowtie \text{Departments})$

Explanation of Common Operations Used in Queries:

- σ (**Select**): Filters rows based on a condition (like $\text{Age} > 30$ or $\text{Dept_Name} = \text{'Sales'}$).
- π (**Project**): Chooses specific columns (like Name or Dept_Name).
- $\bowtie$ (**Join**): Combines two relations based on a common attribute (like Dept_ID).
- $\cup$ (**Union**): Combines two relations and removes duplicates.
- $-$ (**Difference**): Returns the rows in the first relation that are not in the second.
- $\cap$ (**Intersection**): Returns the common rows between two relations.

SQL FUNDAMENTALS : REFER LAB MANUAL AND OBSERVATION